

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №4
“Шаблоны проектирования и модульное тестирование в Python”

Выполнил:
Студент группы ИУ5-34Б
Кострыкина Е.Г.
Преподаватель:
Нардид А.Н.

Москва 2025

Задание

1. Необходимо для произвольной предметной области реализовать от одного до трех шаблонов проектирования: один порождающий, один структурный и один поведенческий. В качестве справочника шаблонов можно использовать следующий каталог. Для сдачи лабораторной работы в минимальном варианте достаточно реализовать один паттерн.
2. В модульных тестах необходимо применить следующие технологии:
 - TDD - фреймворк.
 - BDD - фреймворк.
 - Создание Mock-объектов.

Код программы

prototype.py

```
# Порождающий шаблон: прототип
import copy
from abc import ABC, abstractmethod

class FlowerPrototype(ABC):
    """Абстрактный базовый класс для клонирования цветов"""
    @abstractmethod
    def clone(self):
        pass

class Flower(FlowerPrototype):
    """Конкретный цветок с возможностью клонирования"""
    def __init__(self, name, color, price):
        self.name = name
        self.color = color
        self.price = price

    def clone(self):
        return copy.deepcopy(self)

    def __str__(self):
        return f"{self.color} {self.name} {self.price}₽"

class BouquetPrototype(FlowerPrototype):
    """Прототип букета (композиции цветов)"""
    def __init__(self, name):
        self.name = name
        self.flowers = []

    def add_flower(self, flower):
        self.flowers.append(flower)

    def get_cost(self):
        return sum(f.price for f in self.flowers)

    def clone(self):
        return copy.deepcopy(self)

    def __str__(self):
        flowers_str = "\n- ".join(str(f) for f in self.flowers)
        return f"Букет '{self.name}' {self.get_cost()}₽:\n- {flowers_str}"
```

decorator.py

```
# Структурный шаблон: декоратор
from abc import ABC, abstractmethod

class OrderComponent(ABC):
    """Абстрактный компонент заказа"""
    @abstractmethod
    def get_description(self):
        pass

    @abstractmethod
```

```

def get_cost(self):
    pass

class BasicOrder(OrderComponent):
    """Базовый заказ"""
    def __init__(self, bouquet):
        self.bouquet = bouquet

    def get_description(self):
        return f"Заказ: {self.bouquet.name}"

    def get_cost(self):
        return self.bouquet.get_cost()

class OrderDecorator(OrderComponent):
    """Базовый декоратор для заказов"""
    def __init__(self, wrapped):
        self._wrapped = wrapped

    def get_description(self):
        return self._wrapped.get_description()

    def get_cost(self):
        return self._wrapped.get_cost()

class WrappingDecorator(OrderDecorator):
    """Декоратор подарочной упаковки"""
    def __init__(self, wrapped, style="стандартная"):
        super().__init__(wrapped)
        self.style = style
        self.price = 50 if style == "стандартная" else 100

    def get_description(self):
        return f"{self._wrapped.get_description()} + упаковка ({self.style})"

    def get_cost(self):
        return self._wrapped.get_cost() + self.price

class CardDecorator(OrderDecorator):
    """Декоратор открытки"""
    def __init__(self, wrapped, message=""):
        super().__init__(wrapped)
        self.message = message
        self.price = 75

    def get_description(self):
        if self.message:
            msg = f"с текстом '{self.message}'"
        else:
            msg = ""
        return f"{self._wrapped.get_description()} + открытка {msg}"

    def get_cost(self):
        return self._wrapped.get_cost() + self.price

```

iterator.py

Поведенческий шаблон: итератор

```

class BouquetIterator:
    """Итератор для букета"""
    def __init__(self, bouquet):
        self._bouquet = bouquet
        self._index = 0

    def __next__(self):
        if self._index < len(self._bouquet.flowers):
            flower = self._bouquet.flowers[self._index]
            self._index += 1
            return flower
        raise StopIteration

    def __iter__(self):
        return self

```

tests/test_mock_iterator.py

```

# Модульное тестирование через создание Mock-объектов
from unittest.mock import Mock
from iterator import BouquetIterator

def test_mock():
    mock_flower = Mock()
    mock_flower.name = "Тестовый цветок"

    mock_bouquet = Mock()
    mock_bouquet.flowers = [mock_flower]

    iterator = BouquetIterator(mock_bouquet)

    result = next(iterator)

    assert result is mock_flower
    assert result.name == "Тестовый цветок"

    print("Тест пройден успешно")

if __name__ == "__main__":
    test_mock()

```

tests/test_tdd_prototype.py

```

# Модульное тестирование через TDD - фреймворк
import sys
import os
from prototype import Flower, BouquetPrototype

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__),
'..')))

class TestBouquetPrototypeTDD:
    def test_bouquet_creation_tdd(self):
        bouquet = BouquetPrototype("Свадебный")

        assert bouquet.name == "Свадебный"
        assert bouquet.flowers == []
        assert bouquet.get_cost() == 0

```

```

def test_add_flower_to_bouquet_tdd(self):
    bouquet = BouquetPrototype("Весенний")
    flower = Flower("Тюльпан", "желтый", 50)

    bouquet.add_flower(flower)

    assert len(bouquet.flowers) == 1
    assert bouquet.flowers[0] is flower

def test_bouquet_cost_calculation_tdd(self):
    bouquet = BouquetPrototype("Дорогой букет")

    assert bouquet.get_cost() == 0

    bouquet.add_flower(Flower("Роза", "красная", 100))
    assert bouquet.get_cost() == 100

    bouquet.add_flower(Flower("Лилии", "белые", 150))
    assert bouquet.get_cost() == 250

    bouquet.add_flower(Flower("Орхидея", "фиолетовая", 300))
    assert bouquet.get_cost() == 550

def test_bouquet_clone_tdd(self):
    original = BouquetPrototype("Оригинальный букет")
    original.add_flower(Flower("Роза", "красная", 100))
    original.add_flower(Flower("Тюльпан", "желтый", 50))

    cloned = original.clone()

    assert cloned.name == "Оригинальный букет"
    assert len(cloned.flowers) == 2
    assert cloned.get_cost() == 150

    assert cloned.flowers[0] is not original.flowers[0]
    assert cloned.flowers[1] is not original.flowers[1]

    cloned.flowers[0].price = 200
    assert original.flowers[0].price == 100

def test_bouquet_str_method_tdd(self):
    bouquet = BouquetPrototype("Тестовый букет")
    bouquet.add_flower(Flower("Роза", "красная", 100))
    bouquet.add_flower(Flower("Тюльпан", "желтый", 50))

    result = str(bouquet)

    assert "Букет 'Тестовый букет' 150Р:" in result
    assert "красная Роза 100Р" in result
    assert "желтый Тюльпан 50Р" in result

def test_new_feature_tdd_example():
    bouquet = BouquetPrototype("Тест")
    bouquet.add_flower(Flower("Роза", "красная", 100))

```

tests/features/decorator.feature

Feature: Декорирование заказов в цветочном магазине

Как клиент цветочного магазина

Я хочу добавлять дополнительные услуги к моему заказу

Чтобы получить персонализированный сервис

Background:

Дано у меня есть базовый букет "Праздничный" стоимостью 500Р

Scenario: Добавление стандартной упаковки к заказу

Дано у меня есть базовый заказ с букетом

Когда я добавляю стандартную упаковку

Тогда общая стоимость заказа должна быть 550Р

И описание должно содержать "упаковка (стандартная)"

Scenario: Добавление праздничной упаковки

Дано у меня есть базовый заказ с букетом

Когда я добавляю праздничную упаковку

Тогда общая стоимость заказа должна быть 600Р

И описание должно содержать "упаковка (праздничная)"

Scenario: Добавление открытки с сообщением

Дано у меня есть базовый заказ с букетом

Когда я добавляю открытку с текстом "С Днем Рождения!"

Тогда общая стоимость заказа должна быть 575Р

И описание должно содержать "открытка с текстом 'С Днем Рождения!'"

Scenario: Добавление открытки без сообщения

Дано у меня есть базовый заказ с букетом

Когда я добавляю открытку

Тогда общая стоимость заказа должна быть 575Р

И описание должно содержать "открытка"

Scenario: Комбинирование упаковки и открытки

Дано у меня есть базовый заказ с букетом

Когда я добавляю стандартную упаковку

И я добавляю открытку с текстом "Поздравляю!"

Тогда общая стоимость заказа должна быть 625Р

И описание должно содержать "упаковка (стандартная)"

И описание должно содержать "открытка с текстом 'Поздравляю!'"

tests/features/steps/decorator_steps.py

```
# Модульное тестирование через BDD - фреймворк
import sys
import os
from prototype import Flower, BouquetPrototype
from decorator import BasicOrder, WrappingDecorator, CardDecorator

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__),
'../../..')))

@given('у меня есть базовый букет "{bouquet_name}" стоимостью {cost}Р')
def step_create_basic_bouquet(context, bouquet_name, cost):
    cost = cost.replace('Р', '')

    context.bouquet = BouquetPrototype(bouquet_name)
    flower = Flower("Цветок", "разноцветный", int(cost))
    context.bouquet.add_flower(flower)

@given('у меня есть базовый заказ с букетом')
def step_create_basic_order(context):
    context.order = BasicOrder(context.bouquet)
```

```

@given('у меня есть базовый заказ с букетом стоимостью {cost}₽')
def step_create_order_with_cost(context, cost):
    cost = cost.replace('₽', '')

    context.bouquet = BouquetPrototype("Букет")
    flower = Flower("Цветок", "Любой", int(cost))
    context.bouquet.add_flower(flower)
    context.order = BasicOrder(context.bouquet)

@when('я добавляю стандартную упаковку')
def step_add_standard_wrapping(context):
    context.order = WrappingDecorator(context.order, "стандартная")

@when('я добавляю праздничную упаковку')
def step_add_premium_wrapping(context):
    context.order = WrappingDecorator(context.order, "праздничная")

@when('я добавляю открытку')
def step_add_card_without_message(context):
    context.order = CardDecorator(context.order)

@when('я добавляю открытку с текстом "{message}"')
def step_add_card_with_message(context, message):
    context.order = CardDecorator(context.order, message)

@when('я добавляю {service}')
def step_add_generic_service(context, service):
    if "стандартную упаковку" in service:
        context.order = WrappingDecorator(context.order, "стандартная")
    elif "праздничную упаковку" in service:
        context.order = WrappingDecorator(context.order, "праздничная")
    elif "открытку" in service and "с текстом" in service:
        import re
        match = re.search(r'"([^\"]*)"', service)
        message = match.group(1) if match else ""
        context.order = CardDecorator(context.order, message)
    elif "открытку" in service:
        context.order = CardDecorator(context.order)

@then('общая стоимость заказа должна быть {expected_cost}₽')
def step_verify_total_cost(context, expected_cost):
    expected_cost = expected_cost.replace('₽', '')

    actual_cost = context.order.get_cost()
    expected = int(expected_cost)

    assert actual_cost == expected, \
        f"Ожидалось {expected}₽, получено {actual_cost}₽"

@then('описание должно содержать "{text}"')
def step_verify_description_contains(context, text):
    description = context.order.get_description()
    assert text in description, \
        f"Текст '{text}' не найден в описании: '{description}'"

@then('общая стоимость должна быть {expected_cost}₽')

```

```
def step_verify_outline_cost(context, expected_cost):
    expected_cost = expected_cost.replace('₽', '')

    actual = context.order.get_cost()
    expected = int(expected_cost)

    assert actual == expected, \
        f"Ожидалось {expected}₽, получено {actual}₽" # python -m behave
tests/features/decorator.feature
```

Работа программы

```
===== test session starts =====
```

```
collecting ... collected 1 item
```

```
test_mock_iterator.py::test_mock PASSED
```

```
[100%]Тест пройден успешно
```

```
===== 1 passed in 0.03s =====
```

```
===== test session starts =====
```

```
collecting ... collected 1 item
```

```
test_tdd_prototype.py::TestBouquetPrototypeTDD::test_bouquet_creation_tdd PASSED [100%]
```

```
===== 1 passed in 0.03s =====
```

```
C:\Users\ekate\PycharmProjects\Kostrykina-CS-Labs-Sem-3>python -m behave tests/features/decorator.feature
```

```
USING RUNNER: behave.runner:Runner
```

```
Feature: Декорирование заказов в цветочном магазине<[90m # tests/features/decorator.feature:1<[0m
```

```
    Как клиент цветочного магазина
```

```
    Я хочу добавлять дополнительные услуги к моему заказу
```

```
    Чтобы получить персонализированный сервис
```

```
Feature: Декорирование заказов в цветочном магазине <[90m # tests/features/decorator.feature:1<[0m
```

```
Scenario: Добавление стандартной упаковки к заказу <[90m # tests/features/decorator.feature:9<[0m
```

```
Scenario: Добавление праздничной упаковки <[90m # tests/features/decorator.feature:15<[0m
```

```
Scenario: Добавление открытки с сообщением <[90m # tests/features/decorator.feature:21<[0m
```

```
Scenario: Добавление открытки без сообщения <[90m # tests/features/decorator.feature:27<[0m
```

```
Scenario: Комбинирование упаковки и открытки <[90m # tests/features/decorator.feature:33<[0m
```

```
1 feature passed, 0 failed, 0 skipped
```

```
5 scenarios passed, 0 failed, 0 skipped
```

```
0 steps passed, 0 failed, 0 skipped
```

```
Took 0min 0.000s
```